

3.1.1 Forward Pass

Unlike a conventional network where the output is the activations from the L^{th} layer, the output of a DEQ is the equilibrium point itself. Therefore, the forward evaluation could be any procedure that solves for this equilibrium point. Conventional deep sequence networks, if they converge to an equilibrium, can be considered a form of *fixed-point iterations*:

$$\mathbf{z}_{1:T}^{[i+1]} = f_{\theta}(\mathbf{z}_{1:T}^{[i]}; \mathbf{x}_{1:T}) \quad \text{for } i = 0, 1, 2, \dots \quad (5)$$

One can alternatively use other methods that provide faster convergence guarantees. For notational convenience, we define g_{θ} and rewrite Eq. (4) as $g_{\theta}(\mathbf{z}_{1:T}^*; \mathbf{x}_{1:T}) = f_{\theta}(\mathbf{z}_{1:T}^*; \mathbf{x}_{1:T}) - \mathbf{z}_{1:T}^* \rightarrow 0$. The equilibrium state $\mathbf{z}_{1:T}^*$ is thus the root of g_{θ} , which we can find more easily with Newton’s method or quasi-Newton methods (e.g., Broyden’s method [10]):

$$\mathbf{z}_{1:T}^{[i+1]} = \mathbf{z}_{1:T}^{[i]} - \alpha B g_{\theta}(\mathbf{z}_{1:T}^{[i]}; \mathbf{x}_{1:T}) \quad \text{for } i = 0, 1, 2, \dots \quad (6)$$

where B is the Jacobian inverse (or its low-rank approximation) at $\mathbf{z}_{1:T}^{[i]}$, and α is the step size. But generally, one can exploit any black-box root-finding algorithm to solve for the equilibrium point in the forward pass, given an initial estimate $\mathbf{z}_{1:T}^{[0]}$ (which we set to $\mathbf{0}$): $\mathbf{z}_{1:T}^* = \text{RootFind}(g_{\theta}; \mathbf{x}_{1:T})$

3.1.2 Backward Pass

A major problem with using a black-box RootFind is that we are no longer able to rely on explicit backpropagation through the exact operations in the forward pass. While one can certainly fix an algorithm (say Newton’s method) to obtain the equilibrium, and then store and backpropagate through all the Newton iterations, we provide below an alternative procedure that is much simpler, requires constant memory, and assumes no knowledge of the black-box RootFind.

Theorem 1. (Gradient of the Equilibrium Model) *Let $\mathbf{z}_{1:T}^* \in \mathbb{R}^{T \times d}$ be an equilibrium hidden sequence with length T and dimensionality d , and $\mathbf{y}_{1:T} \in \mathbb{R}^{T \times q}$ the ground-truth (target) sequence. Let $h : \mathbb{R}^d \rightarrow \mathbb{R}^q$ be any differentiable function and let $\mathcal{L} : \mathbb{R}^q \times \mathbb{R}^q \rightarrow \mathbb{R}$ be a loss function (where $h, \mathcal{L}$ are applied in a vectorized manner) that computes*

$$\ell = \mathcal{L}(h(\mathbf{z}_{1:T}^*), \mathbf{y}_{1:T}) = \mathcal{L}(h(\text{RootFind}(g_{\theta}; \mathbf{x}_{1:T})), \mathbf{y}_{1:T}). \quad (7)$$

Then the loss gradient w.r.t. $(\cdot)$ (for instance, θ or $\mathbf{x}_{1:T}$) is

$$\frac{\partial \ell}{\partial (\cdot)} = - \frac{\partial \ell}{\partial \mathbf{z}_{1:T}^*} (J_{g_{\theta}}^{-1} \big|_{\mathbf{z}_{1:T}^*}) \frac{\partial f_{\theta}(\mathbf{z}_{1:T}^*; \mathbf{x}_{1:T})}{\partial (\cdot)} = - \frac{\partial \ell}{\partial h} \frac{\partial h}{\partial \mathbf{z}_{1:T}^*} (J_{g_{\theta}}^{-1} \big|_{\mathbf{z}_{1:T}^*}) \frac{\partial f_{\theta}(\mathbf{z}_{1:T}^*; \mathbf{x}_{1:T})}{\partial (\cdot)}, \quad (8)$$

where $J_{g_{\theta}}^{-1} \big|_{\mathbf{x}}$ is the inverse Jacobian of g_{θ} evaluated at $\mathbf{x}$.

The proof is provided in Appendix A. The insight provided by Theorem 1 is at the core of our method and its various benefits. Importantly, the backward gradient through the “infinite” stacking can be represented as one step of matrix multiplication that involves the Jacobian at equilibrium. For instance, an SGD update step on model parameters θ would be

$$\theta^+ = \theta - \alpha \cdot \frac{\partial \ell}{\partial \theta} = \theta + \alpha \frac{\partial \ell}{\partial \mathbf{z}_{1:T}^*} (J_{g_{\theta}}^{-1} \big|_{\mathbf{z}_{1:T}^*}) \frac{\partial f_{\theta}(\mathbf{z}_{1:T}^*; \mathbf{x}_{1:T})}{\partial \theta}. \quad (9)$$

Note that this result is independent of the root-finding algorithm we choose or the internal structure of the transformation f_{θ} , and thus does not require any storage of the intermediate hidden states, which is necessary for backpropagation in conventional deep networks.

3.1.3 Accelerating DEQ by Approximating the Inverse Jacobian

One challenge of enforcing the forward and backward passes described in Sections 3.1.1 and 3.1.2 is the cost of computing the exact inverse Jacobian $J_{g_{\theta}}^{-1}$ at every intermediate Newton iteration. We propose to address this using Broyden’s method [10], a quasi-Newton approach that makes low-rank updates to approximate $J_{g_{\theta}}^{-1}$ via the Sherman-Morrison formula [42]:

$$J_{g_{\theta}}^{-1} \big|_{\mathbf{z}_{1:T}^{[i+1]}} \approx B_{g_{\theta}}^{[i+1]} = B_{g_{\theta}}^{[i]} + \frac{\Delta \mathbf{z}^{[i+1]} - B_{g_{\theta}}^{[i]} \Delta g_{\theta}^{[i+1]}}{\Delta \mathbf{z}^{[i+1] \top} B_{g_{\theta}}^{[i]} \Delta g_{\theta}^{[i+1]}} \Delta \mathbf{z}^{[i+1] \top} B_{g_{\theta}}^{[i]}, \quad (10)$$

where $\Delta \mathbf{z}^{[i+1]} = \mathbf{z}_{1:T}^{[i+1]} - \mathbf{z}_{1:T}^{[i]}$ and $\Delta g_\theta^{[i+1]} = g_\theta(\mathbf{z}_{1:T}^{[i+1]}; \mathbf{x}_{1:T}) - g_\theta(\mathbf{z}_{1:T}^{[i]}; \mathbf{x}_{1:T})$. Initially, we set $B_{g_\theta}^{[0]} = -I$ and the Broyden iterations are stopped when either the norm of $g_\theta^{[i]}$ falls below a tolerance ε or when the maximum number of iterations is reached. This lets us avoid the cubic cost induced by the inverse operation.

A similar idea can be used for the backward pass as well. Specifically, to compute $-\frac{\partial \ell}{\partial \mathbf{z}_{1:T}^*} (J_{g_\theta}^{-1} |_{\mathbf{z}_{1:T}^*})$ in Theorem 1, we can alternatively solve the linear system

$$(J_{g_\theta}^\top |_{\mathbf{z}_{1:T}^*}) \mathbf{x}^\top + \left(\frac{\partial \ell}{\partial \mathbf{z}_{1:T}^*} \right)^\top = \mathbf{0}, \quad (11)$$

where the first term (a vector-Jacobian product) can be efficiently computed via autograd packages (e.g., PyTorch [45]) for any $\mathbf{x}$, without explicitly writing out the Jacobian matrix. Such linear systems can generally be solved by any *indirect* methods that leverage fast matrix-vector products; we thus propose to also rely on Broyden’s method (other indirect methods would also suffice) to solve for Eq. (11) and directly backpropagate through the equilibrium by Theorem 1 in the backward pass.

3.2 Properties of Deep Equilibrium Models

Section 3.1 develops a sequence model that, while still based on the deep learning philosophy, is quite different from other approaches in the field, as its output is agnostic to the choice of the RootFind algorithm in the forward pass. We now discuss some implications of the DEQ approach.

Memory cost of DEQ. An important benefit of DEQ is its extreme memory efficiency. As outlined in Section 3.1.3, since we are able to use any root-finding algorithm for both the forward and backward passes (e.g., Broyden’s method [10]), a DEQ only needs to store $\mathbf{z}_{1:T}^*$ (the equilibrium sequence), $\mathbf{x}_{1:T}$ (input-related, layer-independent variables), and f_θ for the backward pass. Note that as we only need the vector-Jacobian product (with dimension $N \times Td$, where N is the minibatch size) in Eq. (11), we never need to explicitly construct the Jacobian $J_{g_\theta}^\top |_{\mathbf{z}_{1:T}^*}$, which could be prohibitively large on long and high-dimensional sequences (with dimension $N \times (Td)^2$). Compared to other deep networks, DEQs therefore offer a constant-memory alternative that enables models that previously required multiple GPUs and other implementation-based techniques (e.g., half-precision or gradient checkpointing [12, 13]) to fit easily into a single GPU.

The choice of f_θ . Our analysis in Sections 3.1.1, 3.1.2, and 3.1.3 is independent of the choice of f_θ , and the same kind of memory benefit is present regardless of the type of f_θ . However, to find the equilibrium in a reliable and efficient manner, generally f_θ needs to be stable and constrained. The two instantiations we provide in Section 4 are examples of stable transformations. (The gated activation in TrellisNet and layer normalization in the transformer constrain the output ranges.)

Stacking the DEQ? A natural question arises: if one DEQ is good, can we get additional benefits by “stacking” DEQs (with potentially *different* classes of transformations)? The answer, somewhat surprisingly, is no, as evidenced by the following theorem, which is proved in Appendix B. The theorem essentially shows that stacking multiple DEQs does not create extra representational power over a single DEQ.

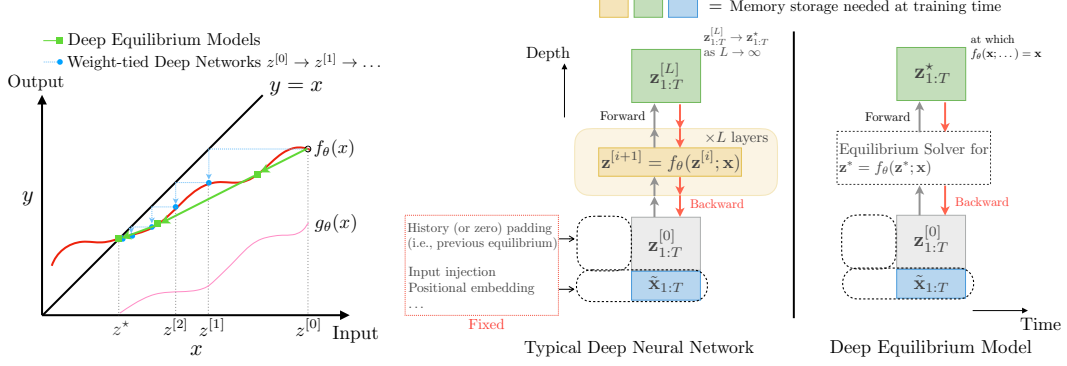
Theorem 2. (Universality of “single-layer” DEQs.) Let $\mathbf{x}_{1:T} \in \mathbb{R}^{T \times p}$ be the input sequence, and $\theta^{[1]}, \theta^{[2]}$ the sets of parameters for stable transformations $f_{\theta^{[1]}} : \mathbb{R}^r \times \mathbb{R}^p \rightarrow \mathbb{R}^r$ and $v_{\theta^{[2]}} : \mathbb{R}^d \times \mathbb{R}^r \rightarrow \mathbb{R}^d$, respectively. Then there exists $\Gamma_\Theta : \mathbb{R}^{d+r} \times \mathbb{R}^p \rightarrow \mathbb{R}^{d+r}$, where $\Theta = \theta^{[1]} \cup \theta^{[2]}$, s.t.

$$\mathbf{z}_{1:T}^* = \text{RootFind}(g_{\theta^{[2]}}^f; \text{RootFind}(g_{\theta^{[1]}}^v; \mathbf{x}_{1:T})) = \text{RootFind}(g_\Theta^\Gamma; \mathbf{x}_{1:T})_{[:, -d:]}, \quad (12)$$

where $[\cdot]_{[:, -d:]}$ denotes the last d feature dimensions of $[\cdot]$.

4 Instantiations of DEQ

While the forward and backward analyses of DEQ do not depend on the internal structure of f_θ , in this section we briefly highlight two examples of f_θ as specific instantiations of DEQ. Both models (TrellisNet [8] and self-attention [48, 18]) achieve state-of-the-art results on various sequence modeling benchmarks. Importantly, through these two very different models and their properties, we illustrate the compatibility of the DEQ approach with all three major families of existing deep sequence networks: *transformers*, *RNNs*, and *temporal convolutional networks (TCNs)*.



(a) A simple illustration of solving for an equilibrium point in 2D. (b) A deep equilibrium model operates with significantly less memory than conventional deep nets due to an analytical backward pass.

Figure 1: Comparison of the DEQ with conventional weight-tied deep networks.

Trellis networks. We briefly introduce the trellis network (TrellisNet) here and refer interested readers to [8] for a detailed description. Generally, TrellisNet is a TCN with two modifications. First, a linear transformation of the original input sequence $\mathbf{x}_{1:T}$ is added to the convolutional outputs at all layers. Second, the convolutional kernel weights are tied across the depth of the network (i.e., TrellisNet is a weight-tied TCN). Thus we can write TrellisNet with convolutional kernel size k , dilation s , and nonlinearity ψ in DEQ form as

$$\begin{aligned}\tilde{\mathbf{x}}_{1:T} &= \text{Input injection (i.e., linearly transformed inputs by } \text{Conv1D}(\mathbf{x}_{1:T}; W_x)) \\ f_\theta(\mathbf{z}_{1:T}; \mathbf{x}_{1:T}) &= \psi(\text{Conv1D}([\mathbf{u}_{-(k-1)s}, \mathbf{z}_{1:T}]; W_z) + \tilde{\mathbf{x}}_{1:T})\end{aligned}$$

where $\mathbf{u}_{-(k-1)s}$ is typically: 1) the last $(k-1)s$ elements of the previous sequence’s output (if using history padding [8]); or 2) simply zero-padding. $[\cdot, \cdot]$ means concatenation along the temporal dimension. Following [8], we use the LSTM gated activation for ψ .

Weight-tied transformers. At a high level, multi-head self-attention transformers [48] are very different from most deep networks. Instead of convolutions or recurrence, a self-attention layer maps the input into Q (query), K (key), and V (value) and computes the attention score between time-steps t_i and t_j as $[QK^\top]_{i,j}$. This attention score is then normalized via softmax and multiplied with the V sequence to produce the output. Since the transformer is order-invariant, prior work proposed to add positional embeddings (PE) [48, 16] to the self-attention operation. Following this design, [18] further proposed the *universal transformer*, which “recurrently stacks” the transformer’s self-attention and transition function block ϕ through a number of layers. Referring readers to [48, 16, 18] for more details, we write a weight-tied transformer in the DEQ form as

$$\begin{aligned}\tilde{\mathbf{x}}_{1:T} &= \text{Input injection (i.e., linearly transformed inputs by } \mathbf{x}_{1:T} W_x) \\ f_\theta(\mathbf{z}_{1:T}; \mathbf{x}_{1:T}) &= \text{LN}(\phi(\text{LN}(\text{SelfAttention}(\mathbf{z}_{1:T} W_{QKV} + \tilde{\mathbf{x}}_{1:T}; \text{PE}_{1:T}))))\end{aligned}$$

where $W_{QKV} \in \mathbb{R}^{d \times 3d}$ produces the Q, K, V for the multi-head self-attention, and LN stands for layer normalization [5]. Note that we add input injection $\tilde{\mathbf{x}}_{1:T}$ to Q, K, V in addition to the positional embedding and initialize with $\mathbf{z}_{1:T}^{[0]} = \mathbf{0}$. Following prior work [48, 19, 16, 18], we use a 2-layer positionwise feedforward residual block for ϕ . In our implementation, we use the memory-augmented transformer proposed by [16], where we feed $[\mathbf{z}_{-T':T}^*, \mathbf{z}_{1:T}]$ (i.e., with history padding of length T') and relative positional embedding $\text{PE}_{-T':T}$ to the self-attention operation.

Figure 1 provides a generic comparison between these conventional weight-tied deep networks and the DEQ approach, highlighting the constant memory requirements of the latter.

5 Experiments

We evaluate DEQ on both synthetic stress tests and realistic large-scale language modeling (where complex long-term temporal dependencies are involved). We use the two aforementioned instantiations of f_θ in DEQ. On both WikiText-103 [35] (which contains $>100\text{M}$ words and a vocabulary size of $>260\text{K}$) and the smaller Penn Treebank corpus (where stronger regularizations are needed for